Lab9 DBSCAN

Source: MLL/Lab9 DBSCAN.ipynb

CELL 1 — CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
```

CELL 2 — CODE

```
import pandas as pd

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00236/seeds_dataset.txt"

columns = [
    "Area",
    "Perimeter",
    "Compactness",
    "Kernel_Length",
    "Kernel_Width",
    "Asymmetry",
    "Groove_Length",
    "Class"
]

df = pd.read_csv(
    url,
    sep=r"\s+",
    header=None,
    names=columns
)

print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)
display(df.head())
```

OUTPUT

```
Dataset loaded successfully!
Shape of dataset: (210, 8)

    Area  Perimeter  Compactness  Kernel_Length  Kernel_Width  Asymmetry  \
0  15.26      14.84       0.8710          5.763         3.312      2.221
1  14.88      14.57       0.8811          5.554         3.333      1.018
2  14.29      14.09       0.9050          5.291         3.337      2.699
3  13.84      13.94       0.8955          5.324         3.379      2.259
4  16.14      14.99       0.9034          5.658         3.562      1.355

   Groove_Length  Class
0          5.220      1
1          4.956      1
2          4.825      1
3          4.805      1
4          5.175      1
```

CELL 3 — CODE

```
df.info()
display(df.describe())
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 210 entries, 0 to 209
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Area           210 non-null    float64
 1   Perimeter      210 non-null    float64
 2   Compactness    210 non-null    float64
 3   Kernel_Length  210 non-null    float64
 4   Kernel_Width   210 non-null    float64
 5   Asymmetry      210 non-null    float64
 6   Groove_Length  210 non-null    float64
 7   Class          210 non-null    int64  
dtypes: float64(7), int64(1)
memory usage: 13.3 KB
```

```
             Area   Perimeter  Compactness  Kernel_Length  Kernel_Width  \
count  210.000000  210.000000   210.000000     210.000000    210.000000   
mean    14.847524   14.559286     0.870999       5.628533      3.258605   
std      2.909699    1.305959     0.023629       0.443063      0.377714   
min     10.590000   12.410000     0.808100       4.899000      2.630000   
25%     12.270000   13.450000     0.856900       5.262250      2.944000   
50%     14.355000   14.320000     0.873450       5.523500      3.237000   
75%     17.305000   15.715000     0.887775       5.979750      3.561750   
max     21.180000   17.250000     0.918300       6.675000      4.033000   

        Asymmetry  Groove_Length       Class  
count  210.000000     210.000000  210.000000  
mean     3.700201       5.408071    2.000000  
std      1.503557       0.491480    0.818448  
min      0.765100       4.519000    1.000000  
25%      2.561500       5.045000    1.000000  
50%      3.599000       5.223000    2.000000  
75%      4.768750       5.877000    3.000000  
max      8.456000       6.550000    3.000000  
```

CELL 4 — CODE

```
print(df.isnull().sum())
```

OUTPUT

```
Area            0
Perimeter       0
Compactness     0
Kernel_Length   0
Kernel_Width    0
Asymmetry       0
Groove_Length   0
Class           0
dtype: int64
```

CELL 5 — CODE

```
X = df.drop("Class", axis=1)
print("Features used for DBSCAN:")
print(X.columns.tolist())
```

OUTPUT

```
Features used for DBSCAN:
['Area', 'Perimeter', 'Compactness', 'Kernel_Length', 'Kernel_Width', 'Asymmetry', 'Groove_Length']
```

CELL 6 — CODE

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Data standardized successfully.")
print("First five standardized observations:")
display(pd.DataFrame(X_scaled, columns=X.columns).head())
```

OUTPUT

```
Data standardized successfully.
First five standardized observations:
```

```
       Area  Perimeter  Compactness  Kernel_Length  Kernel_Width  Asymmetry  \
0  0.142098   0.215462     0.000061       0.304218      0.141702  -0.986152   
1  0.011188   0.008224     0.428515      -0.168625      0.197432  -1.788166   
2 -0.192067  -0.360201     1.442383      -0.763637      0.208048  -0.667479   
3 -0.347091  -0.475333     1.039381      -0.688978      0.319508  -0.960818   
4  0.445257   0.330595     1.374509       0.066666      0.805159  -1.563495   

   Groove_Length  
```

```
0      -0.383577
1      -0.922013
2      -1.189192
3      -1.229983
4      -0.475356
```

CELL 7 — CODE

```
dbscan = DBSCAN(
eps=0.8,
min_samples=5
)
labels = dbscan.fit_predict(X_scaled)
# Add cluster labels to the dataset
df["Cluster"] = labels
print("DBSCAN clustering completed.")
print("Unique labels:", np.unique(labels))
```

OUTPUT

```
DBSCAN clustering completed.
Unique labels: [-1  0  1  2]
```

CELL 8 — CODE

```
cluster_counts = df["Cluster"].value_counts().sort_index()
print("Number of points in each cluster:")
print(cluster_counts)
noise_count = int(np.sum(labels == -1))
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print("\nNumber of clusters:", n_clusters)
print("Number of noise points:", noise_count)
```

OUTPUT

```
Number of points in each cluster:
Cluster
-1     59
 0    107
 1     15
 2     29
Name: count, dtype: int64

Number of clusters: 3
Number of noise points: 59
```

CELL 9 — CODE

```
noise_points = df[df["Cluster"] == -1]
print("Noise Points:")
display(noise_points)
```

OUTPUT

```
Noise Points:
```

	Area	Perimeter	Compactness	Kernel_Length	Kernel_Width	Asymmetry	\
4	16.14	14.99	0.9034	5.658	3.562	1.3550	
8	16.63	15.46	0.8747	6.053	3.465	2.0400	
9	16.44	15.25	0.8880	5.884	3.505	1.9690	
12	13.89	14.02	0.8880	5.439	3.199	3.9860	
15	14.59	14.28	0.8993	5.351	3.333	4.1850	
16	13.99	13.83	0.9183	5.119	3.383	5.2340	
17	15.69	14.75	0.9058	5.527	3.514	1.5990	
18	14.70	14.21	0.9153	5.205	3.466	1.7670	
23	12.08	13.23	0.8664	5.099	2.936	1.4150	
30	13.16	13.82	0.8662	5.454	2.975	0.8551	
35	16.12	15.00	0.9000	5.709	3.485	2.2700	
36	16.20	15.27	0.8734	5.826	3.464	2.8230	
37	17.08	15.38	0.9079	5.832	3.683	2.9560	
39	14.28	14.17	0.8944	5.397	3.298	6.6850	
43	15.50	14.86	0.8820	5.877	3.396	4.7110	
51	15.78	14.91	0.8923	5.674	3.434	5.5930	
52	14.49	14.61	0.8538	5.715	3.113	4.1160	
57	14.92	14.43	0.9006	5.384	3.412	1.1420	
59	12.11	13.47	0.8392	5.159	3.032	1.5020	
60	11.42	12.86	0.8683	5.008	2.850	2.7000	
61	11.23	12.63	0.8840	4.902	2.879	2.2690	
64	12.78	13.57	0.8716	5.262	3.026	1.1760	
77	20.71	17.23	0.8763	6.579	3.814	4.4510	
78	18.94	16.49	0.8750	6.445	3.639	5.0640	
79	17.12	15.55	0.8892	5.850	3.566	2.8580	
80	16.53	15.34	0.8823	5.875	3.467	5.5320	
81	18.72	16.19	0.8977	6.006	3.857	5.3240	
82	20.20	16.89	0.8894	6.285	3.864	5.1730	

87	18.98	16.66	0.8590	6.549	3.670	3.6910
88	21.18	17.21	0.8989	6.573	4.033	5.7800
89	20.88	17.05	0.9031	6.450	4.032	5.0160
90	20.10	16.99	0.8746	6.581	3.785	1.9550
93	18.59	16.05	0.9066	6.037	3.860	6.0010
94	18.36	16.52	0.8452	6.666	3.485	4.9330
97	18.98	16.57	0.8687	6.449	3.552	2.1440
98	18.17	16.26	0.8637	6.271	3.512	2.8530
100	16.41	15.25	0.8866	5.718	3.525	4.2170
108	19.94	16.92	0.8752	6.675	3.763	3.2520
113	19.14	16.61	0.8722	6.259	3.737	6.6820
114	20.97	17.25	0.8859	6.563	3.991	4.6770
120	20.24	16.91	0.8897	6.315	3.962	5.9010
124	15.99	14.89	0.9064	5.363	3.582	3.3360
128	20.16	17.03	0.8735	6.513	3.773	1.9100
129	17.55	15.66	0.8991	5.791	3.690	5.3660
134	15.56	14.89	0.8823	5.776	3.408	4.9720
135	15.38	14.66	0.8990	5.477	3.465	3.6000
137	15.57	15.15	0.8527	5.920	3.231	2.6400
138	15.60	15.11	0.8580	5.832	3.286	2.7250
141	13.32	13.94	0.8613	5.541	3.073	7.0350
142	13.34	13.95	0.8620	5.389	3.074	5.9950
146	11.43	13.13	0.8335	5.176	2.719	2.2210
151	12.01	13.52	0.8249	5.405	2.776	6.9920
165	12.10	13.15	0.8793	5.105	2.941	2.2010
179	12.21	13.47	0.8453	5.357	2.893	1.6610
184	12.89	13.77	0.8541	5.495	3.026	6.1850
188	11.23	12.82	0.8594	5.089	2.821	7.5240
192	11.87	13.02	0.8795	5.132	2.953	3.5970
203	12.70	13.41	0.8874	5.183	3.091	8.4560
207	13.20	13.66	0.8883	5.236	3.232	8.3150

	Groove_Length	Class	Cluster
4	5.175	1	-1
8	5.877	1	-1
9	5.533	1	-1
12	4.738	1	-1
15	4.781	1	-1
16	4.781	1	-1
17	5.046	1	-1
18	4.649	1	-1
23	4.961	1	-1
30	5.056	1	-1
35	5.443	1	-1
36	5.527	1	-1
37	5.484	1	-1
39	5.001	1	-1
43	5.528	1	-1
51	5.136	1	-1
52	5.396	1	-1
57	5.088	1	-1
59	4.519	1	-1
60	4.607	1	-1
61	4.703	1	-1
64	4.782	1	-1
77	6.451	2	-1
78	6.362	2	-1
79	5.746	2	-1
80	5.880	2	-1
81	5.879	2	-1
82	6.187	2	-1
87	6.498	2	-1
88	6.231	2	-1
89	6.321	2	-1
90	6.449	2	-1
93	5.877	2	-1
94	6.448	2	-1
97	6.453	2	-1
98	6.273	2	-1
100	5.618	2	-1
108	6.550	2	-1
113	6.053	2	-1
114	6.316	2	-1
120	6.188	2	-1
124	5.144	2	-1
128	6.185	2	-1
129	5.661	2	-1
134	5.847	2	-1
135	5.439	2	-1
137	5.879	2	-1
138	5.752	2	-1
141	5.440	3	-1
142	5.307	3	-1
146	5.132	3	-1
151	5.270	3	-1
165	5.056	3	-1
179	5.178	3	-1

```
184          5.316      3       -1
188          4.957      3       -1
192          5.132      3       -1
203          5.000      3       -1
207          5.056      3       -1
```

CELL 10 — CODE

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained variance ratio:")
print(pca.explained_variance_ratio_)
print("Total variance represented:",
round(pca.explained_variance_ratio_.sum(), 4))
```

OUTPUT

```
Explained variance ratio:
[0.71874303 0.17108184]
Total variance represented: 0.8898
```

CELL 11 — CODE

```
plt.figure(figsize=(10, 7))
unique_labels = sorted(set(labels))
for label in unique_labels:
  cluster_points = X_pca[labels == label]
if label == -1:
  marker = "x"
  size = 80
  label_name = "Noise"
else:
  marker = "o"
  size = 60
  label_name = f"Cluster {label}"
plt.scatter(
cluster_points[:, 0],
cluster_points[:, 1],
marker=marker,
s=size,
label=label_name
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("DBSCAN Clustering of Seeds Dataset")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT

```
<Figure size 1000x700 with 1 Axes>
```

CELL 12 — CODE

```
non_noise = labels != -1
if n_clusters >= 2 and np.sum(non_noise) > n_clusters:
  score = silhouette_score(
  X_scaled[non_noise],
  labels[non_noise]
  )
  print("Silhouette Score:", round(score, 4))
else:
  print("Silhouette Score cannot be calculated because DBSCAN")
  print("produced fewer than two valid clusters.")
```

OUTPUT

```
Silhouette Score: 0.3755
```

CELL 13 — CODE

```
comparison = pd.crosstab(
df["Class"],
df["Cluster"]
)
print("Actual Class vs DBSCAN Cluster:")
display(comparison)
```

OUTPUT

```
Actual Class vs DBSCAN Cluster:
```

```
Cluster  -1   0   1   2
Class
1        22  48   0   0
2        26   0  15  29
3        11  59   0   0
```

CELL 14 — CODE

```
print("=" * 55)
print("DBSCAN CLUSTERING SUMMARY")
print("=" * 55)
print("Total observations :", len(df))
print("Number of features :", X.shape[1])
print("Number of clusters :", n_clusters)
print("Number of noise points :", noise_count)
print("\nCluster-wise distribution:")
print(cluster_counts)
print("=" * 55)
```

OUTPUT

```
=======================================================
DBSCAN CLUSTERING SUMMARY
=======================================================
Total observations : 210
Number of features : 7
Number of clusters : 3
Number of noise points : 59

Cluster-wise distribution:
Cluster
-1     59
 0    107
 1     15
 2     29
Name: count, dtype: int64
=======================================================
```

CELL 15 — CODE

```
eps_values = [0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
results = []
for eps in eps_values:
  model = DBSCAN(eps=eps, min_samples=5)
  temp_labels = model.fit_predict(X_scaled)
  temp_clusters = len(set(temp_labels)) - (1 if -1 in temp_labels else 0)
  temp_noise = int(np.sum(temp_labels == -1))
  results.append({
  "eps": eps,
  "clusters": temp_clusters,
  "noise_points": temp_noise
  })
results_df = pd.DataFrame(results)
display(results_df)
```

OUTPUT

```
   eps  clusters  noise_points
0  0.5         1           205
1  0.6         5           161
2  0.7         7            78
3  0.8         3            59
4  0.9         1            28
5  1.0         1            13
```